

Single Sign-On (SSO) Strategies

Java Backend Interview Reference

What Is SSO?

Single Sign-On allows a user to authenticate once and gain access to multiple applications without re-entering credentials. The Identity Provider (IdP) authenticates the user; Service Providers (SP) / Relying Parties (RP) trust the IdP's assertion.

Term	Definition
Identity Provider (IdP)	Central system that authenticates users and issues tokens/assertions. Examples: Keycloak, Okta, Azure AD, Google, Auth0
Service Provider (SP)	Application relying on the IdP for authentication. Also called Relying Party (RP) in OIDC.
Assertion / Token	Proof of authentication issued by IdP: SAML Assertion (XML) or OIDC ID Token (JWT)
SSO Session	Session at the IdP level — user logged in to IdP. Separate from each SP's session.
Federation	Cross-organizational SSO: Company A's users can log in to Company B's app via their own corporate IdP.

SSO Protocol 1: OAuth2 + OIDC (Modern Standard)

What OIDC Adds to OAuth2

OAuth2 is an authorization protocol (access tokens for API calls). OIDC (OpenID Connect) adds authentication on top — it specifies an ID Token (JWT) that contains user identity claims (sub, name, email, etc.).

```
// OIDC Token Types: // 1. ID Token (JWT): contains user identity. Consumed by the APP, not the API. // {"iss":"https://idp.example.com","sub":"user123","name":"Alice","email":"alice@co.com","exp":...} // 2. Access Token: used to call APIs. May be JWT or opaque. // 3. Refresh Token: renew access token without re-login. //
OIDC Authorization Code Flow: // 1. User clicks Login -> app redirects to IdP: // GET https://idp/authorize?client_id=app&response_type=code&scope=openid email profile // &redirect_uri=https://app/callback&state=random&nonce=random // 2. User authenticates at IdP (if not already SSO-logged-in) // 3. IdP redirects back: GET https://app/callback?code=AUTH_CODE&state=random // 4. App exchanges code: // POST /token {code, client_id, client_secret, redirect_uri, grant_type=authorization_code} // 5. IdP returns: {id_token, access_token, refresh_token} // 6. App validates ID token (signature, iss, aud, exp, nonce)
```

Spring Boot OAuth2 Login (OIDC)

```
# application.yml spring: security: oauth2: client: registration: keycloak: client-id: my-app client-secret: secret authorization-grant-type: authorization_code scope: openid,profile,email provider: keycloak: issuer-uri: https://keycloak.example.com/realms/myrealm # Auto-discovers: /authorize, /token, /userinfo, /jwks.json @Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { return http . oauth2Login(o -> o .defaultSuccessUrl("/dashboard", true) .failureUrl("/login?error") .userInfoEndpoint(u -> u.oidcUserService(oidcUserService())) ) .build(); }
```

SSO Protocol 2: SAML 2.0 (Enterprise Legacy)

How SAML Works

SAML uses XML-based assertions. Used widely in enterprise environments (corporate Active Directory federation). More complex than OIDC but many enterprise apps still require it.

```
// SAML SP-Initiated Flow: // 1. User accesses app (SP) // 2. SP creates SAML AuthnRequest, redirects user to IdP: // GET https://idp/saml/sso?SAMLRequest=BASE64(DEFLATE(XML)) // 3. IdP authenticates user (or reuses existing SSO session) // 4. IdP POSTs SAML Response to SP (Assertion Consumer Service URL): // POST https://app/saml/acs // Body: SAMLResponse=BASE64(SIGNED_XML) // 5. SP validates: signature, conditions (notBefore/notOnOrAfter), audience, issuer // 6. SP creates local session for user // SAML Assertion contains: // alice@co.com // ADMIN
```

Attribute	OIDC	SAML 2.0
Format	JSON / JWT	XML
Transport	HTTP redirects + Bearer tokens	HTTP POST / Redirect bindings
Complexity	Low — well-supported libraries	High — XML parsing, schema validation

Attribute	OIDC	SAML 2.0
Mobile support	Excellent	Poor (browser redirects only)
Adoption trend	Growing — modern standard	Legacy enterprise; stable installed base
IdP examples	Auth0, Okta, Google, Keycloak	ADFS, Okta, Shibboleth, Ping Identity
Use when	New projects, consumer-facing, mobile	Enterprise with existing ADFS/LDAP; legal/compliance requirement

SSO Protocol 3: Kerberos (Internal Network)

Kerberos is used for on-premise corporate network authentication (Windows domain). Users log in to their workstation; apps on the same network accept Kerberos tickets without re-login.

Step	Detail
1. AS Request	Client requests a Ticket Granting Ticket (TGT) from the Authentication Server using their password
2. TGT issued	AS returns encrypted TGT (encrypted with user's key). User caches TGT.
3. Service Ticket	Client sends TGT to Ticket Granting Server (TGS), requests service ticket for a specific app
4. App access	Client sends service ticket to app. App decrypts with shared secret from KDC. No password sent.
Java usage	SPNEGO/Negotiate filter in Spring Security. Browser sends Kerberos ticket automatically in corporate AD environments.

SSO Architecture Patterns

Centralized IdP (Most Common)

User -> App A -> Redirect to IdP -> Authenticate -> Token -> App A (SSO session at IdP) User -> App B -> Redirect to IdP -> Already logged in -> Token issued -> App B (no re-authentication)

Microservices SSO — API Gateway Pattern

Client --[Bearer Token]--> API Gateway | +-----+-----+-----+ | | | Service A Service B Service C
(validate JWT locally from IdP's JWKS) // Each service validates JWT independently: // 1. Fetch JWKS from IdP:
GET https://idp/.well-known/jwks.json // 2. Cache public keys // 3. Verify JWT signature, iss, aud, exp on
each request // No call to IdP per request – fast!

Token Exchange (RFC 8693) — Service-to-Service with User Context

// Service A receives user token, needs to call Service B on behalf of user // Token Exchange: exchange user token for service-scoped token POST /token grant_type=urn:ietf:params:oauth2:grant-type:token-exchange subject_token=USER_ACCESS_TOKEN subject_token_type=urn:ietf:params:oauth2:token-type:access_token requested_token_type=urn:ietf:params:oauth2:token-type:access_token audience=service-b // Returns new token scoped for service-b with user identity preserved

Single Logout (SLO)

SSO makes login easy, but logout must be global. Single Logout ensures logging out of one app logs the user out of all apps sharing the SSO session.

Mechanism	How It Works	Challenge
OIDC Front-Channel Logout	IdP loads logout URLs from all SPs as iframes/images in browser	Unreliable — iframes blocked, ad blockers
OIDC Back-Channel Logout	IdP sends HTTP POST to each SP's logout endpoint directly (server-to-server) with logout token	More reliable. SP must invalidate local sessions on receiving logout token.
SAML Single Logout	SP sends LogoutRequest to IdP; IdP sends LogoutRequest to all other SPs; all respond with LogoutResponse	Complex multi-party XML exchange; partial logout risk
Short session TTL	Each SP session expires quickly (e.g., 15 min). User effectively logged out soon after IdP session ends.	User experience: re-authentication required frequently

Keycloak — Common Production Setup

```
// Keycloak is the most popular open-source IdP for Java shops // Realms: separate namespaces (one per environment or tenant) // Clients: each SP registers as a client with client_id + secret // Roles: realm roles and client-specific roles // application.yml for Keycloak + Spring Boot spring: security: oauth2: resourceserver: jwt: issuer-uri: https://keycloak.example.com/realm/my-realm # Fetches JWKS from issuer-uri/.well-known/openid-configuration // Extract Keycloak roles from JWT @Bean public JwtAuthenticationConverter keycloakConverter() { JwtGrantedAuthoritiesConverter conv = new JwtGrantedAuthoritiesConverter(); conv.setAuthoritiesClaimName("realm_access.roles"); // Keycloak puts roles here conv.setAuthorityPrefix("ROLE_"); JwtAuthenticationConverter jac = new JwtAuthenticationConverter(); jac.setJwtGrantedAuthoritiesConverter(conv); return jac; }
```

Interview Q&A;

Q1: What is the difference between OAuth2 and OIDC?

OAuth2 is an authorization framework — it allows an app to get access tokens to call APIs on behalf of a user. It does NOT define how to authenticate users. OIDC (OpenID Connect) is a thin identity layer on top of OAuth2 that adds an ID Token (JWT with user claims like sub, name, email) and a /userinfo endpoint. If you need authentication (login), use OIDC. If you only need to authorize API access, OAuth2 alone is enough.

Q2: Explain the OIDC Authorization Code + PKCE flow step by step.

1. App generates code_verifier (random), computes code_challenge = SHA256(verifier). 2. Redirect user to IdP with code_challenge. 3. User authenticates. 4. IdP returns authorization code to redirect_uri. 5. App POSTs code + code_verifier to /token endpoint. 6. IdP verifies SHA256(verifier) == code_challenge, returns id_token + access_token. PKCE prevents code interception — stolen code is useless without verifier.

Q3: What is the difference between SAML and OIDC?

SAML uses XML assertions and HTTP POST/Redirect bindings. It's the older enterprise standard, integrated with corporate Active Directory via ADFS. OIDC uses JSON/JWT and is simpler, mobile-friendly, and REST-native. New projects should use OIDC. Choose SAML only when integrating with legacy enterprise systems that require it (government, old corporate portals).

Q4: How does SSO work when a user logs out? What is Single Logout?

When a user logs out of one SP, the SP should also notify the IdP to end the SSO session, and the IdP should notify all other SPs. OIDC Back-Channel Logout: IdP sends an HTTP POST with a logout token to each SP's registered back-channel logout endpoint; each SP invalidates its local session. Front-channel logout (browser iframes) is unreliable. A practical fallback: use short SP session TTLs (15-30 min) so logout propagates naturally.

Q5: How do you validate an ID Token received from an IdP?

1. Verify signature using IdP's public keys from JWKS endpoint. 2. Verify iss matches the expected IdP. 3. Verify aud matches your client_id. 4. Verify exp hasn't passed. 5. Verify iat is reasonable (not too old). 6. Verify nonce matches the one sent in the authorization request (prevents replay). 7. For SAML: also verify Conditions NotBefore/NotOnOrAfter, audience restriction. Never skip any step.

Q6: How do you implement SSO across multiple microservices?

All services act as OAuth2 Resource Servers that validate JWTs from the shared IdP. The API Gateway validates tokens for inbound requests. Services fetch the IdP's JWKS and cache public keys for offline validation — no call to IdP per request. The JWT carries user identity and scopes. For service-to-service calls with user context: use token exchange (RFC 8693) or forward the user token with reduced scope. Service-to-service without user: Client Credentials flow.

Q7: What is federation and cross-organizational SSO?

Federation allows users from Organization A to log into Organization B's applications using their own corporate IdP. Example: a contractor from CompanyA accesses CompanyB's portal. CompanyB's IdP (SP IdP) trusts CompanyA's IdP (user's IdP) via a federation agreement. Implemented with SAML federation (metadata exchange) or OIDC federation. AWS Cognito, Okta, and Azure AD all support B2B federation.

Q8: What is a Keycloak Realm and when would you use multiple realms?

A Keycloak Realm is a completely isolated namespace with its own users, clients, roles, and identity providers. Use one realm per environment (dev/staging/prod) or one realm per large enterprise tenant (when tenants need completely separate user stores, SSO boundaries, and branding). Don't create a realm per small tenant — manage them within one realm using groups and client-specific roles. Realm isolation is stricter than role/group isolation within a realm.